# Experiment No. 6

**Student Name: Debanuj Kumar De** **UID:** 23BCS12048
**Branch:** BE CSE **Section/Group:** KRG_2-B
**Semester:** 6th **Subject Code:** 23CSH-314
**Subject Name:** System Design **Date of Performance:** 25/02/2026

## 1. Aim:

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

## 2. Objective

1. Understand the architecture of a massive-scale media streaming system.
2. Identify functional requirements such as direct-to-cloud uploads and personalized feeds.
3. Identify non-functional requirements including CDN caching, high throughput, and latency constraints.
4. Analyze the decoupling of synchronous API requests using event-driven stream processing.
5. Design REST APIs for video management, search, and user interactions.

## 3. Procedure

1. Studied real-world video streaming architectures and content delivery networks.
2. Identified core entities such as Users, Videos, Interactions, and View Histories.
3. Analyzed the Pre-Signed URL pattern to bypass API Gateway bottlenecks for large file uploads.
4. Collected functional and non-functional requirements for media delivery.
5. Designed APIs for video processing, metadata retrieval, and social engagement.
6. Evaluated streaming protocols (HLS/DASH) and separated thumbnail storage for optimization.
7. Analyzed stream processing (Apache Flink/Kafka) for real-time, high-volume view aggregation

# 4. System Requirements:

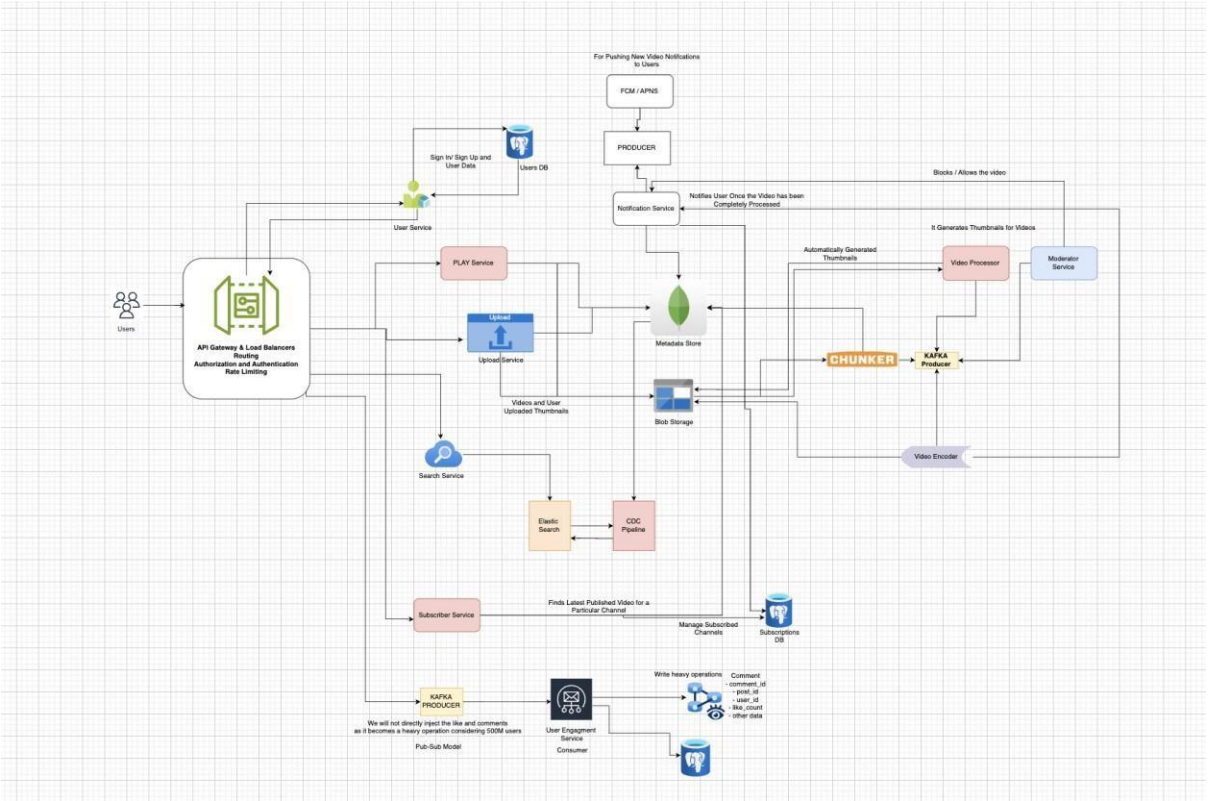
## A. Functional Requirements

- Users can upload videos.
- Users can watch videos.
- Users can search for videos.
- Users can like/unlike videos and post comments with real-time count updates.
- The system supports search, recommendations, and channel subscriptions.
- Users can sign up, log in, and manage their channels and uploaded content.

## B. Non-Functional Requirements

- The system must scale to handle millions of concurrent viewers and uploads.
- Video playback should start quickly and switch quality without noticeable buffering. Even at low bandwidth.
- The platform must maintain high availability (99.9% uptime or higher).
- Likes and comments should update dynamically with low latency.

# 5. High Level Design (HLD):



## 6. Learning Outcomes (What I Have Learnt)

- Designed a highly scalable, event-driven video streaming architecture.
- Identified critical functional and non-functional requirements for media delivery.
- Designed REST APIs and decoupled asynchronous processing pipelines.
- Understood the application of CDNs, adaptive bitrate streaming, and stream processing for massive-scale systems.